These context objects are powerful and are the key to advanced filter behaviors like short-circuiting the pipeline and handling exceptions. We'll make use of them in chapter 22 when we create more complex filter examples.

The async version of the resource filter requires implementing a single method, as shown in listing 21.2. As for the sync version, you're passed a `ResourceExecutingContext` object as an argument, and you're passed a delegate representing the remainder of the filter pipeline. You must call this delegate (asynchronously) to execute the remainder of the pipeline, which returns an instance of `ResourceExecutedContext`.

Listing 21.2 Example resource filter implementing `IAsyncResourceFilter`

```
public class LogAsyncResourceFilter : Attribute, IAsyncResourceFilter
{
    public async Task OnResourceExecutionAsync(                    ❶
        ResourceExecutingContext context,
        ResourceExecutionDelegate next)                            ❷
    {
        Console.WriteLine("Executing async!");                     ❸
        ResourceExecutedContext executedContext = await next();    ❹
        Console.WriteLine("Executed async!");                      ❺
    }
}
```

❶ Executed at the start of the pipeline, after authorization filters

❷ You're provided a delegate, which encapsulates the remainder of the filter pipeline.

❸ Called before the rest of the pipeline executes

❹ Executes the rest of the pipeline and obtains a ResourceExecutedContext instance

❺ Called after the rest of the pipeline executes

The sync and async filter implementations have subtle differences, but for most purposes they're identical. I recommend implementing the sync version for simplicity, falling back to the async version only if you need to.